



UNIVERSITÉ DE
MONTPELLIER



Partitions satisfaisantes dans les graphes

Travail d'Étude et de Recherche — Master 1 Algorithmique

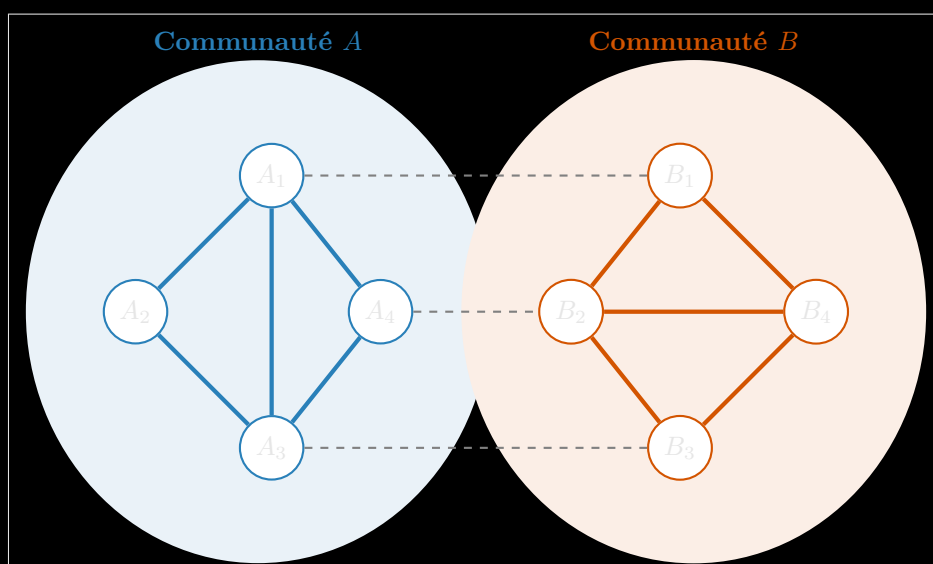


FIGURE 1 – Représentation d'une partition satisfaisante (A, B) : chaque sommet possède strictement plus de connexions internes (lignes pleines) que de connexions externes (lignes pointillées).

Auteur

Ivan LEJEUNE

Encadrant

Stéphane BESSY

25 mai 2026

1 Introduction et définitions formelles

1.1 Contexte et définitions

Soit $G = (V, E)$ un graphe simple, non orienté et fini. L'objet de ce rapport est l'étude du problème de l'existence d'une *partition satisfaisante* de ses sommets, un concept introduit pour formaliser la notion de cohésion interne au sein de sous-structures de graphes.

Définition 1 Degrés interne et externe.

Soit $G = (V, E)$ un graphe, (A, B) une partition de V (avec $A, B \neq \emptyset$, $A \cap B = \emptyset$ et $A \cup B = V$), et $x \in V$.

- Si $x \in A$, son **degré interne** est $d_{\text{int}}(x) = |N(x) \cap A|$ et son **degré externe** est $d_{\text{ext}}(x) = |N(x) \cap B|$.
- Si $x \in B$, son **degré interne** est $d_{\text{int}}(x) = |N(x) \cap B|$ et son **degré externe** est $d_{\text{ext}}(x) = |N(x) \cap A|$.

Définition 2 Partition satisfaisante.

Une partition (A, B) de V est dite **satisfaisante** si pour tout sommet $x \in V$, on a :

$$d_{\text{int}}(x) \geq d_{\text{ext}}(x)$$

Un sommet vérifiant cette propriété est dit *satisfait*. Le problème de décision associé, consistant à déterminer si un graphe admet une telle partition, est noté SATISFACTORY GRAPH PARTITION.

Remarque.

Intuitivement, le degré interne d'un sommet mesure combien de ses voisins appartiennent à la même partie de la partition que lui, tandis que le degré externe mesure combien de ses voisins appartiennent à l'autre partie.

Une partition satisfaisante impose que chaque sommet ait au moins autant de voisins dans sa propre partie que dans l'autre, ce qui formalise une notion de « cohésion » ou « communauté » au sein du graphe.

Dans la suite de ce rapport, on partira du principe que le lecteur est familier avec les notions de base de la théorie des graphes, telles que les cycles, les arbres, les graphes réguliers entres autres.

1.2 Objectifs du TER et attendus

L'objectif principal de ce TER est d'attaquer une conjecture sur l'existence de partitions satisfaisantes dans les graphes k -réguliers (on donnera plus de détails sur cette conjecture plus tard dans le rapport). A titre personnel, je pense que cette conjecture est vraie et ma recherche s'est donc portée sur la construction de graphes k -réguliers en essayant de les empêcher d'admettre une partition satisfaisante.

Les attendus de ce TER sont les suivants :

- Une revue de la littérature sur les partitions satisfaisantes et les graphes k -réguliers. Cela permet notamment d'avoir le contexte nécessaire pour comprendre les enjeux de la conjecture et les méthodes utilisées pour l'attaquer.
- Des résultats théoriques sur les conditions d'existence de partitions satisfaisantes dans les graphes k -réguliers. Il s'agit notamment de démontrer les propriétés au début de la section 4.
- Du code permettant d'expérimenter avec les graphes k -réguliers. Ce code doit permettre de tester l'existence et trouver des partitions satisfaisantes pour des graphes de petite taille, ainsi que de visualiser les résultats obtenus.

2 Motivations

Le problème de la recherche d'une partition satisfaisante trouve des échos directs dans l'analyse des structures relationnelles modernes.

En **recherche de communautés** (réseaux sociaux ou biologiques), les sommets représentent des individus ou des entités biologiques (gènes, protéines) et les arêtes correspondent à leurs interactions. Une partition satisfaisante isole des sous-groupes stables où l'activité endogène prédomine sur les échanges exogènes. Cela permet d'identifier des communautés d'intérêt, comme des groupes d'amis dans un réseau social ou des groupes de gènes fonctionnellement liés dans un réseau biologique.

En **théorie des jeux et sociologie**, ce problème modélise la formation d'alliances défensives. Un groupe de sommets A forme une alliance stable si chaque membre trouve au moins autant de soutien au sein de A qu'il n'affronte de menaces à l'extérieur (B). Cette modélisation s'applique historiquement aux équilibres géopolitiques et industriels (fusions et alliances stratégiques).

Enfin, en **intelligence artificielle**, le partitionnement de graphes est un pilier de l'apprentissage non supervisé (*clustering*). Garantir qu'un cluster possède une densité interne supérieure à sa connectivité externe améliore la robustesse des algorithmes de classification textuelle et de vision par ordinateur.

L'étude de ce problème, à la fois théorique et appliquée, est donc cruciale pour comprendre les liens complexes dans les réseaux modernes et pour développer des outils d'analyse dans des domaines aussi variés que la sociologie, la biologie et l'informatique.

3 État de l'art

Bien que le partitionnement de graphes soit un domaine classique de l'optimisation combinatoire, l'étude spécifique des partitions satisfaisantes est plus récente. Notre analyse s'appuie principalement sur les travaux fondateurs de Bazgan, Tuza et Vanderpooten (2010) [2], l'état de l'art des alliances défensives par Yero et Rodríguez-Velázquez (2018) [4], ainsi que les algorithmes structurels présentés par Gaikwad, Maity et Tripathi (2020) [3].

Nos connaissances sur la complexité du problème se résument ainsi :

- **NP-complétude** : Déterminer si un graphe quelconque admet une partition satisfaisante est NP-complet [4].
- **Cas polynomiaux** : Le problème devient polynomial pour les graphes à clique-width bornée [3] et pour les graphes de degré maximal restreint ($\Delta(G) \leq 4$) [2].

D'autres travaux ont exploré des variantes du problème, notamment les partitions satisfaisantes avec des contraintes supplémentaires sur les tailles des parties, les degrés des sommets, ou même le nombre de parties dans la partition. Cependant, la question de l'existence d'une partition satisfaisante pour les graphes k -réguliers, en particulier pour $k \geq 5$, reste largement ouverte et constitue l'une des principales motivations de notre étude.

D'autre part, beaucoup de résultats présents dans ces papiers s'intéressent à trouver des bornes sur certaines propriétés des graphes qui garantissent ou empêchent l'existence d'une partition satisfaisante. Dans ce papier, on s'est plus concentré sur la manière d'en trouver si il en existe une et sur la construction de graphes qui n'en admettent pas pour attaquer la conjecture au début de la section suivante.

4 Résultats pour les graphes k -réguliers ($k \leq 4$)

On s'intéresse dans cette section à l'existence de partitions satisfaisantes dans les graphes k -réguliers, c'est-à-dire les graphes où chaque sommet a exactement k voisins.

On va montrer quelques résultats fondamentaux pour étudier la conjecture suivante :

Conjecture 1. Pour tout entier k , tous les graphes k -réguliers, sauf un nombre fini d'exceptions, admettent une partition satisfaisante.

4.1 Réduction aux graphes connexes

Proposition 1. Tout graphe non connexe admet une partition satisfaisante.

Démonstration.

Soit $G = (V, E)$ un graphe non connexe.

Il existe au moins une composante connexe $A \subset V$ telle que $B = V \setminus A \neq \emptyset$. Considérons la partition (A, B) .

Par définition d'une composante connexe, il n'existe aucune arête entre A et B . Ainsi, pour tout $x \in A$, $d_{\text{ext}}(x) = 0$.

Comme le degré d'un sommet est toujours positif, on a pour tout $x \in A$:

$$d_{\text{int}}(x) \geq 0 = d_{\text{ext}}(x).$$

Le même raisonnement s'applique à tout $y \in B$, où :

$$d_{\text{ext}}(y) = 0 \leq d_{\text{int}}(y).$$

La partition (A, B) est donc satisfaisante. □

4.2 Analyse systématique par degré

La conjecture étant ouverte pour les graphes k -réguliers avec $k \geq 5$, on propose d'étudier au cas par cas les graphes k -réguliers pour $k \leq 4$ afin d'obtenir des résultats de base et de mieux comprendre les mécanismes à l'œuvre dans la formation de partitions satisfaisantes.

4.2.1 Le cas $k = 1$

Un graphe 1-régulier connexe est isomorphe à K_2 . Il n'admet aucune partition puisque séparer les deux sommets produit deux ensembles de degré interne nul et externe égal à 1.

4.2.2 Le cas $k = 2$

Un graphe 2-régulier connexe est un cycle C_n .

Si $n = 3$, G est isomorphe à K_3 et n'admet pas de partition satisfaisante.

Si $n \geq 4$, la partition :

$$(\{x, y\}, V \setminus \{x, y\})$$

où $(x, y) \in E$ est satisfaisante.

En effet, pour tout sommet z de G :

- si $z \in \{x, y\}$, alors :

$$d_{\text{int}}(z) = 1 \quad \text{et} \quad d_{\text{ext}}(z) = 1;$$

- si $z \in V \setminus \{x, y\}$, alors :

$$d_{\text{int}}(z) \geq 1 \quad \text{et} \quad d_{\text{ext}}(z) \leq 1.$$

Ainsi, tous les sommets vérifient $d_{\text{int}}(z) \geq d_{\text{ext}}(z)$ et la partition est satisfaisante.

4.2.3 Le cas $k = 3$

Soit G un graphe 3-régulier. On raisonne sur la taille du plus petit cycle de G .

Soit C le plus petit cycle de G .

Supposons que C est de longueur 3, c'est-à-dire que C est un triangle.

Le nombre d'arêtes sortant de C est alors exactement 3, car G est 3-régulier.

- Si un sommet $x \in G \setminus C$ est relié à tous les sommets de C , alors G est isomorphe à K_4 et n'admet pas de partition satisfaisante (voir figure 2).

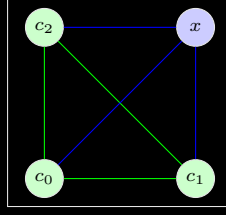


FIGURE 2 – Graphe G avec un 3-cycle C et un 3-sommet x

En effet, il n'y a plus d'arêtes sortant de $C \cup \{x\}$ et G est connexe, donc :

$$G \setminus (C \cup \{x\}) = \emptyset.$$

- Si un sommet $x \in G \setminus C$ est relié à exactement deux sommets de C , alors l'ensemble :

$$A = C \cup \{x\}$$

possède exactement deux arêtes sortantes.

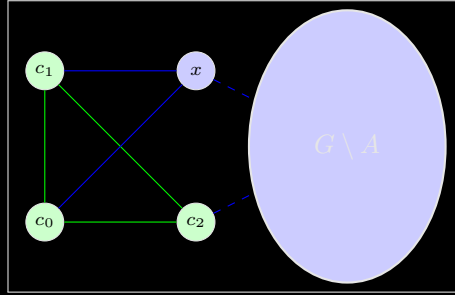


FIGURE 3 – Graphe G avec un 3-cycle C et un 2-sommet x

Si aucun sommet de $G \setminus A$ n'est relié à deux sommets de A , alors tous les sommets sont satisfaits pour la partition $(A, V \setminus A)$ (voir figure 4).

Sinon, on ajoute ce sommet à A et on obtient encore une partition satisfaisante (voir figure 5) :

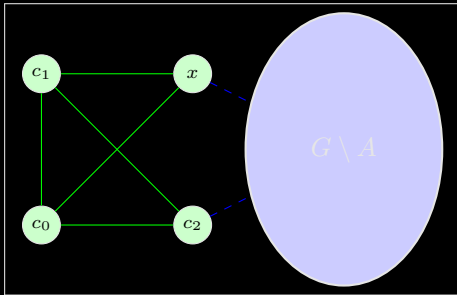


FIGURE 4 – Graphe G avec un 3-cycle C et un 2-sommet x puis rien

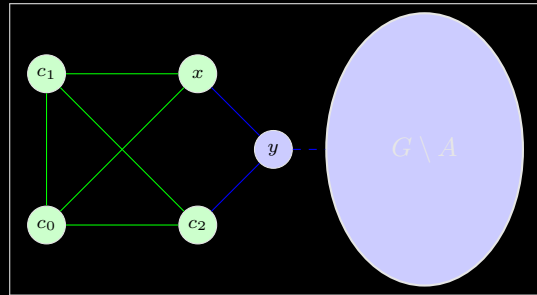


FIGURE 5 – Graphe G avec un 3-cycle C et un 2-sommet x puis un 2-sommet y

- Si tout sommet de $G \setminus C$ est relié à au plus un sommet de C , alors $G \setminus C$ est non vide et tous les sommets sont satisfaits pour la partition $(C, V \setminus C)$ (voir figure 6).

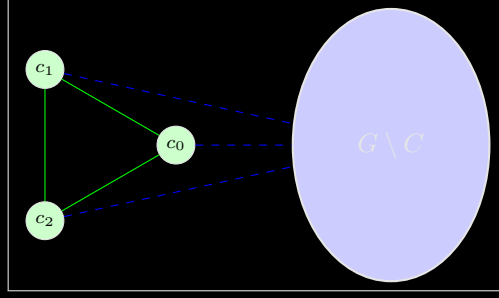


FIGURE 6 – Graphe G avec 3-cycle C

Supposons maintenant que C est de longueur au moins 4.

- Si un sommet de $G \setminus C$ est relié à trois sommets de C , alors on peut former un cycle strictement plus petit que C , ce qui contredit la minimalité de C (voir figure 7).

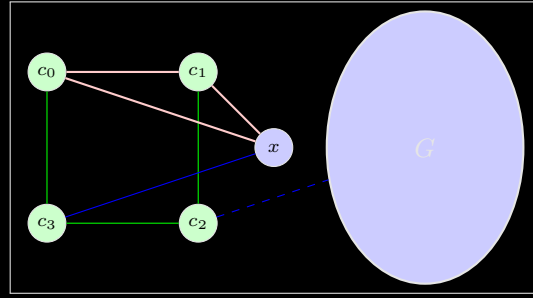


FIGURE 7 – Graphe G avec un 4-cycle C et un 3-sommet x

Donc tout sommet de $G \setminus C$ est relié à au plus deux sommets de C .

- Si tout sommet de $G \setminus C$ est relié à au plus un sommet de C , alors la partition $(C, V \setminus C)$ est satisfaisante (même cas que la figure 6).
- Si un sommet de $G \setminus C$ est relié à exactement deux sommets de C , alors :
 - ces deux sommets doivent être opposés dans C , sinon on obtiendrait un cycle plus petit ;
 - s'il existe un second sommet vérifiant la même propriété, alors soit :

$$G \simeq K_{3,3},$$

qui n'admet pas de partition satisfaisante (voir figure 8), soit on obtient une partition satisfaisante en ajoutant ces deux sommets à C (voir figure 9).

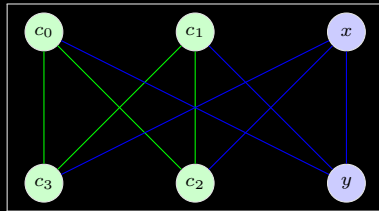


FIGURE 8 – Graphe G avec un 4-cycle C et deux 2-sommets voisins x et y

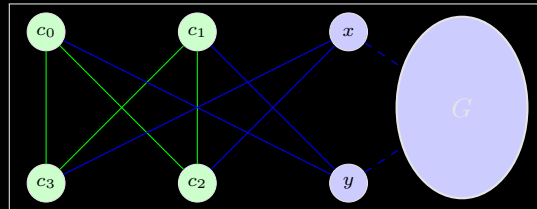


FIGURE 9 – Graphe G avec un 4-cycle C et deux 2-sommets x et y

On voit aussi que pour $|C| \geq 4$, on tombera toujours dans le cas de la figure 9 après avoir rajouté un certain nombre de sommets à C .

- sinon, $G \setminus C$ est non vide et il existe encore une partition satisfaisante.

4.2.4 Le cas $k = 4$: preuve par recherche locale

Pour les graphes 4-réguliers, on utilise une approche un peu différente. On va d'abord commencer par prouver le lemme suivant :

Lemme 2. Tout graphe 4-régulier connexe, à l'exception du graphe complet K_5 , contient au moins deux cycles sommet-disjoints.

Démonstration. Soit $G = (V, E)$ un graphe 4-régulier connexe. Si G est isomorphe à K_5 , il possède exactement 5 sommets. Comme un cycle simple requiert au moins 3 sommets, K_5 ne peut pas contenir deux cycles sommets-disjoints (car $3 + 3 = 6 > 5$).

Supposons désormais que G n'est pas isomorphe à K_5 . Le graphe G étant fini et contenant des cycles (car il est connexe et 4-régulier), on peut définir C comme un cycle de longueur minimale dans G . On note $c = |C|$.

Par minimalité, C est un cycle induit sans corde : chaque sommet de C possède exactement 2 voisins dans C , et donc exactement $4 - 2 = 2$ voisins dans $V \setminus C$. Le nombre total d'arêtes entre C et $V \setminus C$ est donc exactement égal à $2c$.

Supposons par l'absurde que $G \setminus C$ ne contient aucun cycle. Alors, le sous-graphe $T = G \setminus C$ est une forêt. Notons $t = |T|$ son nombre de sommets.

Dans une forêt à t sommets, le nombre d'arêtes internes est au plus $t - 1$. En appliquant le lemme des poignées de main au sous-graphe T , la somme des degrés internes $d_{\text{int}}(u)$ au sein de la forêt vérifie :

$$\sum_{u \in T} d_{\text{int}}(u) = 2|E(T)| \leq 2t - 2 \quad (1)$$

Par ailleurs, chaque sommet $u \in T$ a un degré total de 4 dans G . Sa contribution au degré interne de T et ses voisins sur C vérifient la relation $d_{\text{int}}(u) + \deg_C(u) = 4$. En sommant cette égalité sur l'ensemble T , on obtient :

$$\sum_{u \in T} d_{\text{int}}(u) = \sum_{u \in T} (4 - \deg_C(u)) = 4t - e(T, C)$$

Où $e(T, C)$ représente le nombre total d'arêtes reliant T et C . Nous avons établi que $e(T, C) = 2c$. Ainsi, la somme des degrés internes vaut exactement $4t - 2c$. En injectant ce résultat dans l'inéquation (1), il vient :

$$4t - 2c \leq 2t - 2 \implies 2t + 2 \leq 2c \implies c \geq t + 1$$

Analysons maintenant la structure de la forêt T . Toute forêt non vide possède au moins une feuille (ou un sommet isolé) u tel que son degré interne dans T vérifie $d_{\text{int}}(u) \leq 1$. Pour un tel sommet, le nombre de connexions vers le cycle C est au moins égal à :

$$\deg_C(u) = 4 - d_{\text{int}}(u) \geq 3$$

Le sommet $u \in T$ possède donc au moins 3 voisins distincts sur le cycle C . Ces 3 voisins découpent le cycle C en trois chemins disjoints dont la somme des longueurs vaut c . Par le principe des tiroirs, l'un de ces chemins possède une longueur au plus égale à $\lfloor c/3 \rfloor$. En reliant les extrémités de ce court chemin via le sommet u , on forme un nouveau cycle dans G dont la longueur est au plus $\lfloor c/3 \rfloor + 2$.

Par minimalité de la longueur de C , ce nouveau cycle ne peut pas être strictement plus court que C , ce qui impose la condition structurelle :

$$c \leq \left\lfloor \frac{c}{3} \right\rfloor + 2$$

Cette inégalité n'est vérifiée que pour $c = 3$ (un triangle). En effet, pour tout $c \geq 4$, on a $\lfloor c/3 \rfloor + 2 < c$.

Si $c = 3$, notre inégalité précédente $c \geq t + 1$ devient $3 \geq t + 1$, ce qui implique $t \leq 2$. Le nombre total de sommets de notre graphe connexe G est alors $|V| = c + t \leq 3 + 2 = 5$. Le seul graphe connexe 4-régulier ayant au maximum 5 sommets est le graphe complet K_5 .

Cela contredit notre hypothèse de départ selon laquelle G n'est pas isomorphe à K_5 . Par conséquent, l'hypothèse d'absurde est fausse : le sous-graphe $G \setminus C$ contient nécessairement au moins un cycle, ce qui garantit l'existence de deux cycles sommets-disjoints dans G . $\square$

Ensuite, on va utiliser ce lemme pour construire un algorithme qui va créer une partition satisfaisante à partir de ces deux cycles sommets-disjoints.

Théorème 3. Tout graphe 4-régulier connexe, à l'exception du graphe complet K_5 , admet une partition satisfaisante.

Démonstration.

Soit $G = (V, E)$ un graphe 4-régulier connexe différent de K_5 .

On sait d'après le lemme précédent que G contient au moins deux cycles sommets-disjoints. Soit C_1 et C_2 ces deux cycles. On peut construire une partition de V en deux ensembles A et B de la manière suivante :

- A contient tous les sommets de C_1 et tous les sommets de G qui sont reliés à au moins deux sommets de C_1 .
- $B = V \setminus A$.

Par construction, tous les sommets de A ont au moins deux voisins dans A (car ils sont soit dans C_1 , soit reliés à au moins deux sommets de C_1). De plus, comme G est 4-régulier, chaque sommet de A a au plus deux voisins dans B . Ainsi, pour tout sommet $x \in A$, on a $d_{\text{int}}(x) \geq 2 \geq d_{\text{ext}}(x)$. Pour les sommets de B , si ils ne sont pas satisfaits on va les ajouter à A et répéter le processus jusqu'à ce que tous les sommets soient satisfaits.

Comme B contient le cycle C_2 qui est disjoint de C_1 , B ne sera jamais vidé par le processus car il y aura toujours au moins les sommets de C_2 qui restent dans B . Par conséquent, le processus s'arrêtera à un moment donné avec une partition (A, B) telle que tous les sommets de A sont satisfaits et tous les sommets de B sont satisfaits.

Ainsi, G admet une partition satisfaisante. $\square$

5 Les cas $k \geq 5$

On a vu dans les sections précédentes que les graphes réguliers de degré 3 et 4 admettent quasiment tous une partition satisfaisante. Il est donc naturel de se demander si cette propriété se maintient pour les graphes réguliers de degré 5 ou plus.

On détaille dans la suite différents résultats liés à cette question.

5.1 Le saut de complexité entre $k = 4$ et $k = 5$

Dans le cas $k = 4$, un des arguments principaux utilisé pour démontrer l'existence d'une partition satisfaisante est l'existence de deux *noyaux*. Plus formellement, on définit un noyau comme suit :

Définition 3. Soit $G = (V, E)$ un graphe et t un entier positif. Un t -**noyau** de G est un ensemble de sommets $N \subseteq V$ tel que pour tout sommet $u \in N$, on a $\deg_N(u) \geq t$.

Le concept de noyau est crucial pour démontrer l'existence de partitions satisfaisantes comme on peut le voir dans la proposition suivante :

Proposition 4. Soit $G = (V, E)$ un graphe k -régulier et $t = \lceil \frac{k}{2} \rceil$. Si G contient deux t -noyaux disjoints, alors G admet une partition satisfaisante.

Démonstration. Supposons que G contient deux t -noyaux disjoints N_1 et N_2 . On peut construire une partition satisfaisante de G à partir de la partition $(N_1, V \setminus N_1)$ en déplaçant itérativement les sommets non satisfaits.

Soit x un sommet non satisfait. En déplaçant x de sa partie actuelle à l'autre partie, x devient satisfait et le nombre d'arêtes traversant la partition diminue d'au moins 1. Plus précisément, si $d_{\text{int}}(x) = d$ et $d_{\text{ext}}(x) > d$ alors $d_{\text{ext}}(x) \geq d + 1$ et en déplaçant x , le nombre d'arêtes traversant la

partition diminue de $d_{\text{ext}}(x) - d_{\text{int}}(x) \geq 1$. Comme il y a un nombre fini d'arêtes dans le graphe, ce processus doit se terminer après un nombre fini d'étapes, et à ce moment-là, tous les sommets seront satisfaits.

De plus, les sommets de N_1 étant satisfaits dans la partition initiale, ils ne deviendront jamais non satisfaits au cours du processus, ce qui garantit que la première partie de la partition finale ne sera jamais vide. De même, les sommets de N_2 étant satisfaits dans la partition initiale, ils ne deviendront jamais non satisfaits, ce qui garantit que la seconde partie de la partition finale ne sera jamais vide.

Les deux parties étant non vides et tous les sommets étant satisfaits, la partition finale est une partition satisfaisante de G . $\square$

Pour $k = 4$, un 2-noyau correspond simplement à un sous-graphe de degré minimum 2, un cycle. Un tel objet est relativement facile à trouver en général et d'autant plus facile à caractériser lors d'une recherche algorithmique.

Cependant, pour $k = 5$, un 3-noyau correspond à un sous-graphe de degré minimum 3, ce qui est beaucoup plus contraignant.

Un tel sous-graphe doit nécessairement contenir une structure plus complexe que les simples cycles, et ces structures sont plus difficiles à identifier et à manipuler dans le cadre d'une preuve d'existence mais surtout dans le cadre d'une recherche algorithmique.

Un résultat important pour la recherche de noyaux dans un graphe k -régulier est le suivant :

Proposition 5. Soit $G = (V, E)$ un graphe k -régulier de taille n et $t = \lceil \frac{k}{2} \rceil$. Si G contient un t -noyau de taille p alors G contient au moins deux t -noyaux disjoints.

Heureusement, un des résultats que nous avons obtenus est que pour un graphe d'une certaine taille, on n'a besoin que d'un seul t -noyau pour garantir l'existence d'une partition satisfaisante car l'existence d'un second noyau est assurée par le nombre d'arêtes du graphe.

5.2 Le cas $k = 6$ et au-delà

Tout comme pour $k = 5$, les cas $k \geq 7$ présentent des difficultés similaires liées à la complexité croissante des noyaux nécessaires pour garantir l'existence d'une partition satisfaisante. Ces cas sont d'ailleurs toujours ouverts au moment de la rédaction de ce rapport.

Cependant, pour $k = 6$, la conjecture est vérifiée pour tous les graphes 6-réguliers à au moins 12 sommets [1].

On ne redémontrera pas ce résultat dans ce rapport mais l'idée de la preuve repose sur un comptage fin des arêtes traversant la partition qui permet d'obtenir l'existence de noyaux disjoints et donc de partitions satisfaisantes.

6 Méthodes de recherche

Dans notre étude du problème de partition satisfaisante on a fait beaucoup de recherche avec plusieurs méthodes différentes. Dans un premier temps, on va montrer le code qu'on a utilisé pour expérimenter et confirmer des résultats théoriques. Puis, on va présenter des différents axes de recherche qu'on a suivis pour essayer de développer le terrain autour de cette conjecture.

6.1 Le code

Tout au long de ce projet, on a développé un code en Python pour expérimenter avec les graphes et les partitions satisfaisantes. Ce code nous a permis de tester différentes hypothèses, de construire des exemples de graphes qui n'admettent pas de partition satisfaisante, et de vérifier la conjecture pour des graphes de petite taille.

On a commencé par créer des algorithmes de base capables de trouver des partitions satisfaisantes pour des graphes de petite taille en utilisant des méthodes différentes.

La première méthode est le brute-force. Etant donné qu'il y a conséquemment plus de graphes admettant une partition satisfaisante que de graphes n'en admettant pas, on peut partir du principe qu'en général, il existe une partition satisfaisante pour un graphe donné et celle-ci peut être trouvée en testant aléatoirement des partitions du graphe :

```
def satisfying_partition(G, max_attempts=100):
    nodes = list(G.nodes())
    n = len(nodes)

    for attempt in range(max_attempts):
        random.shuffle(nodes)
        A = set(nodes[:n//2])
        B = set(nodes[n//2:])

        improved = True
        while improved:
            improved = False
            for v in list(G.nodes()):
                if v in A:
                    d_int = sum(1 for u in G.neighbors(v) if u in A)
                    d_ext = sum(1 for u in G.neighbors(v) if u in B)
                    if d_ext > d_int and len(A) > 1:
                        A.remove(v)
                        B.add(v)
                        improved = True
                        break
                else:
                    d_int = sum(1 for u in G.neighbors(v) if u in B)
                    d_ext = sum(1 for u in G.neighbors(v) if u in A)
                    if d_ext > d_int and len(B) > 1:
                        B.remove(v)
                        A.add(v)
                        improved = True
                        break

            if is_satisfying(G, A, B):
                return A, B

    return A, B
```

Cela marche bien en pratique mais dans la théorie, il n'y a aucune garantie que ce processus va trouver une partition satisfaisante en un temps raisonnable. Alors, on a décidé de coder un algorithme plus intelligent qui implémente le processus décrit lors des preuves dans les sections précédentes. L'idée est de construire une partition en partant d'un noyau et en ajoutant les sommets

un par un en respectant les conditions de la partition satisfaisante. Voici une implémentation de cette idée :

```
def satisfying_partition_from_cycles(G, C1, C2):
    A = set(C1)
    B = set(G.nodes()) - A

    locked = set(C1) | set(C2)

    improved = True
    while improved:
        improved = False

        for v in G.nodes():
            if v in locked:
                continue

            if v in A:
                d_int = sum(1 for u in G.neighbors(v) if u in A)
                d_ext = sum(1 for u in G.neighbors(v) if u in B)

                if d_ext >= 2: # since 3-regular
                    A.remove(v)
                    B.add(v)
                    improved = True
                    break
            else:
                d_int = sum(1 for u in G.neighbors(v) if u in B)
                d_ext = sum(1 for u in G.neighbors(v) if u in A)

                if d_ext >= 2:
                    B.remove(v)
                    A.add(v)
                    improved = True
                    break

    return A, B
```

Cette fois-ci, on a une garantie que si une partition satisfaisante existe, elle sera trouvée en un temps raisonnable. Cependant, ce code cache un autre problème : comment trouver les cycles à lui donner en entrée ?

Ce code bien qu'adaptable pour un graphe k -régulier quelconque ne marche en pratique que pour $k \leq 4$ car trouver les noyaux nécessaires à lui donner n'est pas une tâche facile.

On utilise le code suivant pour trouver un plus petit cycle car un cycle quelconque ne suffit pas :

```
def shortest_cycle(G):
    import collections

    best_cycle = None
    best_length = float('inf')

    for start in G.nodes():
        dist = {start: 0}
        parent = {start: None}
        queue = collections.deque([start])

        while queue:
            v = queue.popleft()
            for u in G.neighbors(v):
                if u not in dist:
                    dist[u] = dist[v] + 1
                    parent[u] = v
```

```

        queue.append(u)
    elif parent[v] != u:
        # found a cycle
        cycle_length = dist[v] + dist[u] + 1

    if cycle_length < best_length:
        # reconstruct cycle
        path_v = []
        x = v
        while x is not None:
            path_v.append(x)
            x = parent[x]

        path_u = []
        x = u
        while x is not None:
            path_u.append(x)
            x = parent[x]

        # find LCA
        set_v = set(path_v)
        lca = next(x for x in path_u if x in set_v)

        cycle = []
        x = v
        while x != lca:
            cycle.append(x)
            x = parent[x]
        cycle.append(lca)

        tmp = []
        x = u
        while x != lca:
            tmp.append(x)
            x = parent[x]

        cycle.extend(reversed(tmp))

        best_cycle = cycle
        best_length = cycle_length

    return best_cycle

```

Ce code n'est pas très efficace mais il suffit étant donné qu'on ne l'utilise que pour des graphes de petite taille.

Enfin, on a aussi développé un code pour générer des graphes aléatoires et tester la conjecture pour ces graphes ainsi que visualiser les partitions satisfaisantes trouvées :

```

def random_3_regular_graph(n):
    if n < 4 or (3 * n) % 2 != 0:
        raise ValueError("No 3-regular graph exists for this n")
    return nx.random_regular_graph(3, n)

def plot_single(G, A, B, pos=None, title="", node_size=600, font_size=12):
    if pos is None:
        pos = nx.spring_layout(G)

    color_map = ["skyblue" if v in A else "salmon" for v in G.nodes()]

    edge_colors = []

```

```

for u, v in G.edges():
    if (u in A and v in A) or (u in B and v in B):
        edge_colors.append("green")
    else:
        edge_colors.append("red")

plt.figure(figsize=(8, 6))
nx.draw(
    G,
    pos,
    with_labels=True,
    node_color=color_map,
    edge_color=edge_colors,
    node_size=node_size,
    font_size=font_size,
    width=2
)

plt.title(title)

```

Et voici quelques autres codes utilisés pour chercher des partitions :

```

def exhaustive_search_partition(G):
    # fix a node in A then try all partitions of the rest
    nodes = list(G.nodes())
    n = len(nodes)
    A = {nodes[0]}
    B = set(nodes[1:])
    for i in range(1, 2**(n-1)):
        A = {nodes[0]} | {nodes[j] for j in range(1, n) if (i & (1 << (j-1))) > 0}
        B = set(nodes) - A
        if is_satisfying(G, A, B):
            return A, B
    return None, None

def all_satisfying_partitions(G):
    nodes = list(G.nodes())
    n = len(nodes)
    if n % 2 != 0:
        raise ValueError("Graph must have an even number of nodes to be k-regular")
    A = {nodes[0]}
    B = set(nodes[1:])
    satisfying_partitions = []
    for i in range(1, 2**(n-1)):
        A = {nodes[0]} | {nodes[j] for j in range(1, n) if (i & (1 << (j-1))) > 0}
        B = set(nodes) - A
        if is_satisfying(G, A, B):
            satisfying_partitions.append((A, B))
    return satisfying_partitions

```

En mettant tous ces codes ensemble, on peut lancer des tests avec des graphes de petite taille et vérifier la conjecture ainsi que l'efficacité de nos algorithmes :

```

G = nx.random_regular_graph(3, 10)
# G = load_graph(21) # load the 21st first graph from file
save_graph(G) # in case it turns out to be interesting

# Fix layout
seed = 42
pos = nx.spring_layout(G, seed=seed)

```

```

# Compute partitions
A1, B1 = satisfying_partition(G)
A2, B2 = satisfying_partition_3_regular(G)
A3, B3 = exhaustive_search_partition(G)
P = all_satisfying_partitions(G)
print(P)
print(f"Found {len(P)} satisfying partitions (with duplicates)")

# Plotting
n_plots = 3
n_cols = 2
n_rows = math.ceil(n_plots / n_cols)

fig, axes = plt.subplots(n_rows, n_cols, figsize=(6*n_cols, 5*n_rows))
axes = axes.flatten()

# Draw all of them on the same layout
plot_comparison(G, A1, B1,
                pos, ax=axes[0],
                title="Heuristic partition")
plot_comparison(G, A2, B2,
                pos, ax=axes[1],
                title="Cycle-based partition")
plot_comparison(G, A3, B3,
                pos, ax=axes[2],
                title="Exhaustive search partition")
fig.delaxes(axes[3])

# Or draw just one
# plot_single(G, A1, B1, pos=pos, title="Heuristic partition")

plt.show()

print("Now showing all partitions:")
n_plots = len(P)
n_cols = 3
n_rows = math.ceil(n_plots / n_cols)
fig, axes = plt.subplots(n_rows, n_cols, figsize=(6*n_cols, 5*n_rows))
axes = axes.flatten()
for i, (A, B) in enumerate(P):
    plot_comparison(G, A, B, pos, ax=axes[i], title=f"Partition {i+1}")
for j in range(i+1, len(axes)):
    fig.delaxes(axes[j])
plt.show()

```

Ce qui nous donne des résultats comme ceci par exemple :

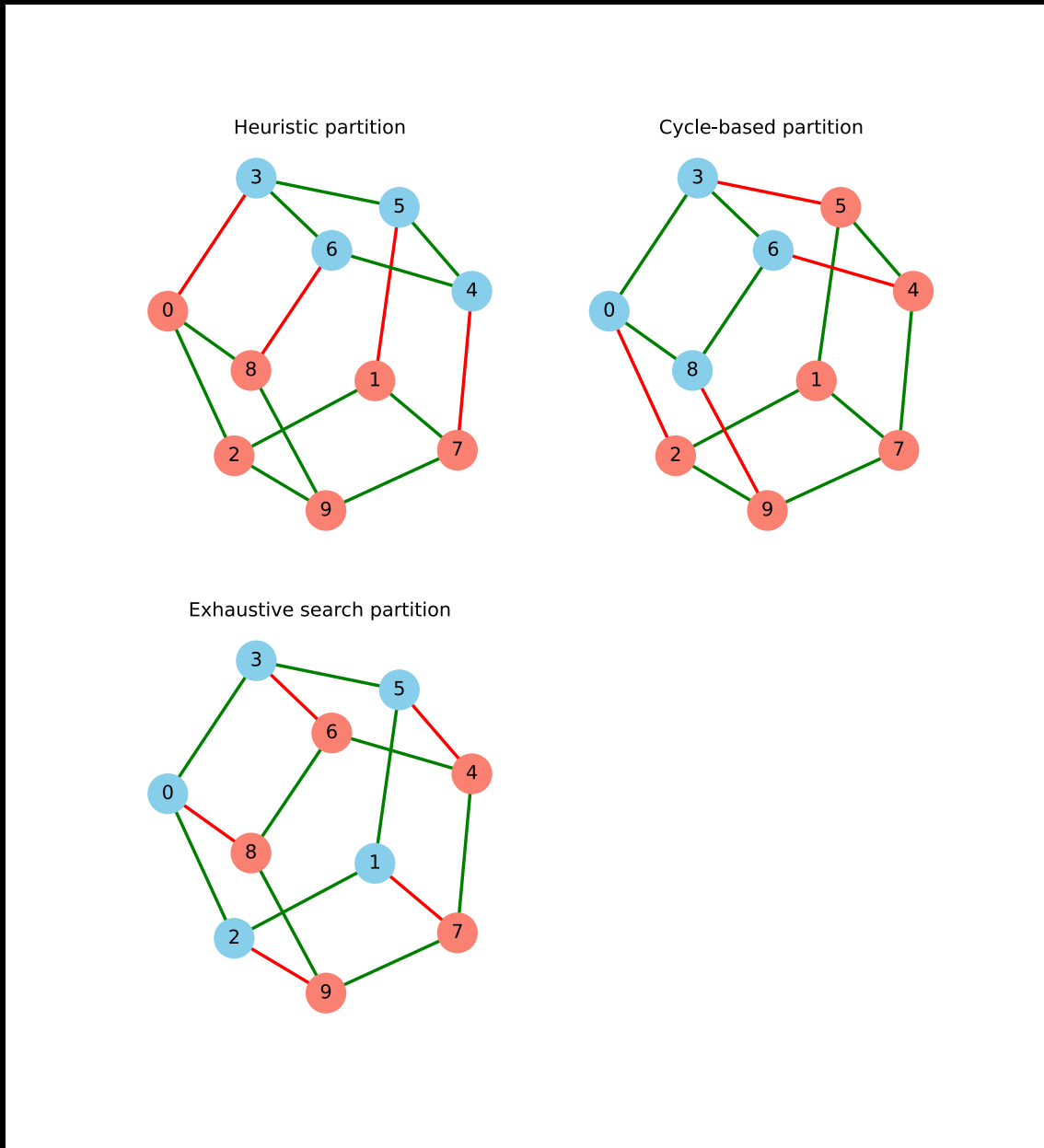


FIGURE 10 – Comparaison de différentes partitions satisfaisantes pour un graphe 3-régulier aléatoire de 10 sommets.

L'intégralité du code source pour ce projet est disponible sur https://github.com/Ivan23BG/TER_M1.

Références

- [1] Pál Bárnkopf, Zoltán Lóránt Nagy, and Zoltán Paulovics. A note on internal partitions : the 5-regular case and beyond. *Graphs and Combinatorics*, 40(2) :36, 2024.
- [2] Cristina Bazgan, Zsolt Tuza, and Daniel Vanderpooten. Satisfactory graph partition, variants, and generalizations. *European Journal of Operational Research*, 206(2) :271–280, 2010.
- [3] Ajinkya Gaikwad, Soumen Maity, and Shuvam Kant Tripathi. The satisfactory partition problem. *arXiv preprint arXiv :2007.14339*, 2020.
- [4] Ismael González Yero and Juan A Rodríguez-Velázquez. Defensive alliances in graphs : a survey. *arXiv preprint arXiv :1308.2096*, 2013.